A SMALL COMPANION FOR THE I LESSONS

I Learn

A tiny I-first path through declarations, data, generics, and reflection.

Open beside the lessons in `src/lessons`. Print it when you want paper margins.

I-FIRST, TINY BY DESIGN

CONTENTS

The Path

This booklet is a map, not the full manual. The real book is the set of tiny programs you can compile, edit, and run again.

1. How To Read A Lesson short loops, real files

2. First Shapes lessons 00-02

3. Data And Callbacks lessons 03-05

4. Libraries And Reflection lessons 07-13

5. The Check Ritual one command

6. The Last Connection lesson 06, saved for last

> The examples use the packaged compiler from `i-windows-x64`, the same way a separate user project should.

CHAPTER 1

How To Read A Lesson

Each lesson is a complete program. Read it once without changing anything, then compile it. After that, make one small change and compile again.

The loop is deliberately plain:

```
 read the I
compile the I
change one thing
compile again
keep the diff tiny
```

A small loop makes the compiler feel like a conversation. If the error message is surprising, the lesson just found the next thing to learn.

Read

Start with names, types, and blocks. Ask what storage exists and who owns it.

Change

Make one edit at a time: a type, a count, a field, a branch, or a generic argument.

Check

Compile the lesson and keep the generated output available for later inspection.

CHAPTER 2

First Shapes

00

Hello World

A complete program is just a declaration named `main` with a procedure type and a block body.

01

Values And Expressions

Typed locals, numeric suffixes, arithmetic, and explicit `cast` calls keep storage and conversion visible.

02

Control Flow

`for`, `if`, `continue`, enum-qualified values, and block-scoped `switch` cases.

> Try changing one return value, one expression, and one enum case. Predict the result before compiling.

```
 name: Type = value;
name: proc(arg: Type)->Return = { ... }
Name: struct = { field: Type; }
```

CHAPTER 3

Data And Callbacks

03

Arrays And Pointers

Fixed arrays make count part of the type. Pointers let procedures mutate storage the caller already owns.

04

Structs And Enums

Structs group fields. Enums name states. Nested field access keeps composition readable.

05

Procedures And Callbacks

Procedure pointer aliases can live in arrays and structs, giving data a callable behavior slot.

Storage

`[4]i32` means exactly four slots. The type carries the count.

Address

`value.&` hands a procedure access to existing storage.

Behavior

`*proc(...)` lets a data structure hold a callable operation.

CHAPTER 4

Libraries, Generics, Reflection

07

Standard Print

`import "std/Print.i"` pulls code from the packaged standard library.

08

Arenas And Array

An arena owns allocation. `Array<i32>` carries length and backing data.

09

Generic Box

`Box<T>` becomes concrete as `Box<i32>`, `Box<f32>`, or another type.

10

Enum-Sized Tables

`Enum<>.value_count` keeps a table sized to the enum declaration.

11

Reflection Metadata

`Payload<>.name`, `field_count`, `size`, and `fields` are generated metadata.

12

Reflection Fields

Walk fields in a loop and read names, types, sizes, pointer depth, and attributes.

13

Metaprogramming Shape

Generic instantiations produce concrete reflected names such as `Slot_i32_reflect`.

CHAPTER 5

The Check Ritual

Use the check script whenever you change a lesson:

```
powershell -ExecutionPolicy Bypass -File scripts/check_all.ps1
```

It finds `I.exe` through `I_HOME`, then `PATH`, then the packaged compiler folder. It compiles every lesson into `build/gen`.

If all is well, it ends with:

```
ok 14 lessons
```

> A learning repo should keep this ritual boring. The interesting part is the source, not the build ceremony.

Related notes: how to read, the shape of I, and reflection and metaprogramming.

CHAPTER 6

The C Connection

Only after the I shape is clear do we name the older neighbor. C is C because it exposes storage, addresses, contiguous arrays, structs as layout, enum states, callable symbols, headers, object files, and platform linkage.

I keeps that physical model visible while making declarations more regular and giving tools generated metadata to work with.

```
 cinclude "stdio.h"
#define I_LEARN_BONUS 7

puts: proc(text: *const char)->i32 = { external; }
```

Lesson 06 shows the boundary: `cinclude` brings in a header, `external` declares a symbol owned elsewhere, and the generated C under `build/gen` shows the platform-facing result.

> Start with I source. Inspect generated C after the shapes make sense.

Related note: the C connection.